# Load Shedding Smart Scheduler

Team Build Guide

Step-by-Step Instructions for Every Team Member

---

**Jayesh | Rajveer | Harshita**

WhatsApp - Gemini AI - EskomSePush - Open-Meteo - FastAPI - Railway

College Project | 100% Free Stack | Zero Hardware | 6-Week Timeline

| 0 Cost | 6 Weeks | 4 Real APIs | 1 Working App |
|---|---|---|---|
| Total Cost | Build Time | Live Data | WhatsApp + AI |

# Table of Contents


| | |
|---|---|
| 01 | Project Overview — What We Are Building |
| 02 | Team Structure & Roles |
| 03 | System Architecture — How Everything Connects |
| 04 | Folder Structure & Setup |
| 05 | Detailed Guide: Jayesh (Backend + Integration Lead) |
| 06 | Detailed Guide: Rajveer (Data + API Engineer) |
| 07 | Detailed Guide: Harshita (AI + User Experience) |
| 08 | Week-by-Week Parallel Timeline |
| 09 | Integration Checklist & Data Format Contract |
| 10 | GitHub Workflow & Team Rules |
| 11 | Deployment on Railway |
| 12 | Demo Script for Viva / Presentation |

# 01 — Project Overview

You are building a **WhatsApp-native AI energy advisor** for South African households. Every morning at 6 AM, your app automatically pulls the real load shedding schedule, checks the solar forecast, reads the household's simulated battery state, and sends a plain-English daily plan over WhatsApp — telling the family exactly what appliances to run and when.

### Sample WhatsApp Message

**What the family receives on WhatsApp at 6 AM:**

```
Good morning! Today's load shedding: 10 AM-12 PM & 6 PM-8 PM (Stage 3).
Battery: 78% | Solar peak: 11 AM-2 PM (est. 2.1 kWh)

My plan for today:
- Run geyser NOW (grid available, battery healthy)
- Washing machine at 1 PM solar window
- Water pump: avoid during cuts

Reply CHANGE, STATUS, or ask me anything.
```

## What is Real vs Simulated?

| Component | Type | Source |
|---|---|---|
| Load shedding schedule | REAL | EskomSePush API |
| Solar / weather forecast | REAL | Open-Meteo API |
| WhatsApp messages | REAL | Twilio Sandbox |
| AI scheduling brain | REAL | Gemini API |
| Inverter + battery data | Simulated | Python class (no hardware needed) |
| Household appliances | Simulated | Hardcoded realistic data |

# 02 — Team Structure & Roles

The project is split into three major pillars. Each team member owns one pillar completely, and work happens in parallel. Integration happens in Weeks 5-6 when Jayesh connects everything.

## Jayesh — Backend + Integration Lead

*"System Architect & Backend Owner"*

- FastAPI backend (api/main.py)
- APScheduler (6 AM automation)
- SQLite database (db/models.py)
- End-to-end pipeline integration
- Deployment on Railway
- GitHub repo management

## Rajveer — Data + API + Simulation Engineer

*"Data Provider Layer (Inputs to AI)"*

- SimulatedInverter class (simulators/inverter.py)
- SimulatedHousehold class (simulators/household.py)
- EskomSePush API integration (integrations/eskomsepush.py)
- Open-Meteo API integration (integrations/openmeteo.py)
- Data format standardization
- API error handling & fallbacks

## Harshita — AI + User Experience

*"AI Brain + User Interaction Layer"*

- Gemini AI agent logic (agent/scheduler.py)
- Prompt engineering & testing
- WhatsApp integration (integrations/whatsapp.py)
- Twilio webhook for user replies
- Conversational reply parsing
- Demo preparation & testing

# 03 — System Architecture

Here is the exact sequence of events that runs every single day. Understand this flow and everything else will make sense.

### 1. 6:00 AM — Scheduler Wakes Up

APScheduler (running inside your FastAPI app on Railway) fires a job every day at 6 AM automatically.

### 2. Fetch Real Outage Schedule (Rajveer)

Calls EskomSePush API with the household's area name. Returns today's outage windows with stage number.

### 3. Fetch Solar Forecast (Rajveer)

Calls Open-Meteo API with latitude/longitude. Returns hourly solar irradiance converted to estimated kWh.

### 4. Read Simulated Inverter State (Rajveer)

SimulatedInverter class generates a morning snapshot: battery %, grid status, solar output.

### 5. Read Household Profile (Rajveer)

SimulatedHousehold class returns the appliance list with wattages, deadlines, and priorities.

### 6. Gemini Agent Reasons (Harshita)

All 4 inputs are sent to Gemini API as a prompt. Gemini produces a WhatsApp-friendly daily plan.

### 7. Save Plan to SQLite (Jayesh)

The generated plan is saved to the SQLite database with a timestamp for history.

### 8. Send WhatsApp Briefing (Harshita)

Twilio API sends the plan as a WhatsApp message. The family wakes up to the plan already in their WhatsApp.

### 9. User Replies — Gemini Parses (Harshita)

If the user replies, Twilio sends it to your webhook. Gemini updates the plan and sends confirmation back.

# 04 — Folder Structure & Setup

Create this exact folder structure before writing any code:

```
smart-scheduler/ simulators/ __init__.py inverter.py <-- Rajveer: SimulatedInverter class household.py <-- Rajveer: SimulatedHousehold class integrations/ __init__.py eskomsepush.py <-- Rajveer: EskomSePush API calls openmeteo.py <-- Rajveer: Open-Meteo API calls whatsapp.py <-- Harshita: Twilio send/receive agent/ __init__.py scheduler.py <-- Harshita: Gemini agent logic api/ __init__.py main.py <-- Jayesh: FastAPI + APScheduler db/ __init__.py models.py <-- Jayesh: SQLite schema .env <-- All API keys (NEVER push to GitHub) requirements.txt <-- Python packages list README.md
```

## requirements.txt — copy this exactly:

```
fastapi uvicorn apscheduler requests python-dotenv google-generativeai twilio sqlalchemy
```

**Setup command:** `pip install -r requirements.txt`

# 05 — Jayesh's Complete Guide

## Backend + Integration Lead

You are the backbone of the project. Your job is to set up the project, build the FastAPI server, wire everything together, and deploy. You start in Week 1 and your heaviest work is in Weeks 5-6.

### Step 1 Project Setup (Week 1)

- Open your terminal / VS Code
- Run: `mkdir smart-scheduler && cd smart-scheduler`
- Create all the folders and __init__.py files shown in Section 04
- Run: `python -m venv venv` (creates virtual environment)
- Activate: `source venv/bin/activate` (Mac/Linux) or `venv\Scripts\activate` (Windows)
- Create requirements.txt with the packages from Section 04
- Run: `pip install -r requirements.txt`
- Create a GitHub repository, invite Rajveer and Harshita as collaborators
- Set up 3 branches: `backend`, `data`, `ai`

### Step 2 Build SQLite Database (Week 2)

- Open `db/models.py`
- Import SQLAlchemy and create a `DailyPlan` model
- Fields: id, date, area, outage_data (JSON string), solar_data (JSON string), plan_text, created_at
- Add a function `save_plan(plan_text, data)` that writes to the DB
- Add a function `get_latest_plan()` that retrieves today's plan
- Test it: create a plan, retrieve it, print it to confirm it works

### Step 3 Build FastAPI Backend (Week 5)

- Open `api/main.py`
- Create a FastAPI app instance
- Create a function `run_daily_job()` that calls all integrations in order:
- (a) Call Rajveer's `get_schedule()` from eskomsepush.py
- (b) Call Rajveer's `get_solar_forecast()` from openmeteo.py
- (c) Call Rajveer's `SimulatedInverter().get_snapshot()`
- (d) Call Rajveer's `SimulatedHousehold().get_appliances()`
- (e) Call Harshita's `generate_plan(outage, solar, inverter, household)`
- (f) Save the plan to SQLite using your `save_plan()`
- (g) Call Harshita's `send_message(phone, plan_text)`
- Add APScheduler: `scheduler.add_job(run_daily_job, 'cron', hour=6, minute=0)`
- Start the scheduler using a FastAPI lifespan event
- Add a `/run-now` GET endpoint that triggers run_daily_job() manually (for demos)
- Add a `/webhook` POST route from Harshita's whatsapp.py
- Run locally: `uvicorn api.main:app --reload`

## Step 4 Integration Testing (Week 5-6)

- Test each module individually by importing and calling it from the Python shell
- Test the full pipeline: call `/run-now` and check if WhatsApp message arrives
- Test the webhook: send a WhatsApp reply and verify Gemini responds correctly
- Fix any data format mismatches between team members' modules
- Run the full flow 3 times and verify consistent results

## Step 5 Deploy to Railway (Week 6)

- Push your code to GitHub (make sure .env is in .gitignore!)
- Go to railway.app and sign up with your GitHub account
- Click 'New Project' and select 'Deploy from GitHub repo'
- Add all .env variables in the Railway Variables tab
- Set Start Command: `uvicorn api.main:app --host 0.0.0.0 --port $PORT`
- Copy your Railway URL and configure Twilio webhook to point to it + /webhook
- Test live: hit your Railway URL's /run-now endpoint from a browser
- Verify the WhatsApp message arrives on a real phone

# 06 — Rajveer's Complete Guide

## Data + API + Simulation Engineer

You provide ALL the data that the AI needs to make decisions. Your modules are the first things that run every morning. If your data is wrong, the entire plan will be wrong. You start in Week 1 and should be done by Week 3 so Harshita can test her AI module with real data.

### Step 1 Build SimulatedInverter (Week 1)

- Open `simulators/inverter.py`
- Create a class `SimulatedInverter`
- In `__init__`, set defaults: battery=78, solar_watts=0, grid_on=True
- Add a method `get_snapshot()` that returns a dictionary with these values
- Make battery drain slightly if grid_on is False (simulates load shedding drain)
- Make solar_watts follow a bell curve: 0 at 6 AM, peaks at 1200W at noon, 0 by 6 PM
- Test it: `python -c 'from simulators.inverter import SimulatedInverter; print(SimulatedInverter().get_snapshot())'`
- Expected output format: `{'battery_pct': 78, 'solar_watts': 0, 'grid_on': True}`

### Step 2 Build SimulatedHousehold (Week 1)

- Open `simulators/household.py`
- Create a class `SimulatedHousehold`
- Add a method `get_appliances()` that returns a list of dicts
- Each appliance has: name, watts, priority (1-5), deadline, flexible (True/False)
- Example: `{'name': 'Geyser', 'watts': 3000, 'priority': 1, 'deadline': '08:00', 'flexible': False}`
- Include at least: Geyser (3kW), Washing Machine (2kW), Fridge (150W), Water Pump (1.5kW), Pool Pump (1.1kW)
- Add a method `get_rules()` that returns: min_battery_reserve: 30, area: 'Cape Town Zone 3'
- Test it the same way as above and verify the output format

### Step 3 Connect EskomSePush API (Week 2)

- Go to eskomsepush.com and sign up — you get a free API key
- Add `ESKOMSEPUSH_KEY=your_key` to the .env file in the root folder
- Open `integrations/eskomsepush.py`
- Use the requests library to call: `https://developer.sepush.co.za/business/2.0/area`
- Pass your API key in the headers and the area ID as a parameter
- Parse the JSON response to extract outage start/end times
- Return a clean list: `[{'start': '10:00', 'end': '12:00', 'stage': 3}, ...]`
- Add error handling: if API fails, return an empty list with a warning
- Test: `python -c 'from integrations.eskomsepush import get_schedule; print(get_schedule())'`

## Step 4 Connect Open-Meteo API (Week 2)

- Open `integrations/openmeteo.py`
- No API key needed — just call the URL directly
- URL: `https://api.open-meteo.com/v1/forecast?latitude=-33.9&longitude=18.4&hourly=shortwave_radiation`
- Change latitude/longitude to your test household location (Cape Town default)
- Parse the hourly shortwave_radiation values (solar energy in W/m2)
- Multiply by 0.0015 (rough panel efficiency) to get estimated kWh per hour
- Return a clean list: `[{'hour': '06:00', 'kwh': 0.0}, {'hour': '12:00', 'kwh': 1.8}, ...]`
- Test it and verify values peak around midday

## Step 5 Data Format Contract (Week 2-3)

- ALL your outputs MUST follow this exact format so Jayesh and Harshita can plug them in:
- `outage: [{'start': '10:00', 'end': '12:00', 'stage': 3}]`
- `solar: [{'hour': '06:00', 'kwh': 0.0}, {'hour': '12:00', 'kwh': 1.8}]`
- `inverter: {'battery_pct': 78, 'solar_watts': 0, 'grid_on': True}`
- `household: [{'name': 'Geyser', 'watts': 3000, ...}]`
- Share sample output files with Harshita by Week 2 so she can start building prompts
- Create a `test_all_data.py` script that calls all 4 functions and prints formatted output

# 07 — Harshita's Complete Guide

## AI + User Experience

You are the intelligence of the system. Your AI agent takes all the raw data and converts it into a smart, human-readable daily plan. You also handle the WhatsApp integration so the family can interact with the system. You start in Week 3 (after Rajveer has sample data ready) and your heaviest work is in Weeks 3-4.

### Step 1 Get Your Gemini API Key (Week 3 — Day 1)

- Go to aistudio.google.com — sign in with your Google account
- Click 'Get API Key' then 'Create API Key'
- Copy the key and add to .env: `GEMINI_KEY=paste_your_key_here`
- Free tier: 1500 requests/day — more than enough for this project
- Test the key works by making a simple API call from Python

### Step 2 Build the Gemini Agent (Week 3)

- Open `agent/scheduler.py`
- Import `google.generativeai` and configure with your key
- Write a function `generate_plan(outage, solar, inverter, household)`
- Build a prompt string that explains the situation to Gemini. Include:
- - Today's outage windows (from outage data)
- - Solar production forecast (from solar data)
- - Current battery level and grid status (from inverter data)
- - List of appliances with their wattages and deadlines (from household data)
- - Rules: minimum battery reserve, priority order
- Ask Gemini to produce a WhatsApp-friendly daily schedule
- Call `model.generate_content(prompt)` and return the text
- Test with fake data first — use hardcoded sample data from Rajveer's format
- Tweak the prompt until the output reads naturally and makes smart decisions
- Test edge cases: What if there are no outages? What if battery is critically low?

### Step 3 Build Reply Parser (Week 3-4)

- Add a function `handle_reply(user_message, current_plan)`
- Send both the user's reply and the current plan to Gemini
- Prompt: 'The user replied: [message]. The current plan is: [plan]. Update the plan or respond.'
- Return the updated plan or response text
- Test with sample replies: 'Change washing machine to 4 PM', 'STATUS', 'What about the pool pump?'

## Step 4 Set Up Twilio WhatsApp (Week 4 — Day 1-2)

- Go to twilio.com and sign up for free
- Go to Console then Messaging then Try it out then Send a WhatsApp message
- Follow instructions to join the Sandbox (you'll send a code to a Twilio number)
- Note your Account SID, Auth Token, and Sandbox number
- Add to .env: `TWILIO_SID=xxx, TWILIO_TOKEN=xxx, TWILIO_FROM=whatsapp:+14155238886`
- NEVER push .env to GitHub!

## Step 5 Build WhatsApp Integration (Week 4)

- Open `integrations/whatsapp.py`
- Import the Twilio Python library
- Write a function `send_message(to, body)` that sends a WhatsApp message
- Write a function `handle_webhook(request)` for incoming replies:
- - Extract the reply text from the Twilio webhook payload
- - Pass it to your `handle_reply()` function from agent/scheduler.py
- - Send the response back via `send_message()`
- Test sending: run your send_message function with a test message
- Test receiving: you'll need Jayesh's FastAPI /webhook route running (Week 5)

## Step 6 Prompt Engineering Tips

- Be specific in your prompt: tell Gemini it is an energy advisor for a South African household
- Include ALL data in the prompt — don't assume Gemini knows anything
- Ask for a structured output: numbered list with time slots and appliance names
- Include the rule about minimum battery reserve (30%) in the prompt
- End the prompt with: 'Format this as a friendly WhatsApp message under 500 characters'
- Test with 5+ different data scenarios before calling it done

# 08 — Week-by-Week Parallel Timeline

This shows what each team member is doing each week. Work happens in parallel — you do NOT need to wait for each other until integration in Week 5.

| Week | Jayesh (Backend) | Rajveer (Data) | Harshita (AI) |
|---|---|---|---|
| 1 | Project setup<br>GitHub repo<br>Folder structure<br>Virtual environment | SimulatedInverter class<br>SimulatedHousehold class<br>Test both with sample data | Learn Gemini API basics<br>Read documentation<br>Test simple API calls |
| 2 | SQLite database<br>db/models.py<br>Test save/retrieve | EskomSePush API integration<br>Open-Meteo API integration<br>Test both APIs<br>Share sample output with team | Study prompt engineering<br>Write first draft of prompt<br>Test with hardcoded data |
| 3 | Review Rajveer's data format<br>Plan API endpoints<br>Start api/main.py skeleton | Polish all 4 data modules<br>Fix any API issues<br>Create test_all_data.py script | Build agent/scheduler.py<br>Full generate_plan() function<br>Test with Rajveer's real data |
| 4 | Continue main.py<br>Add /run-now endpoint<br>Add /webhook route | API refinement<br>Error handling<br>Fallback data if API fails | Twilio WhatsApp setup<br>Build whatsapp.py<br>Test send/receive messages |
| 5 | Connect ALL modules in main.py<br>APScheduler setup<br>Full end-to-end testing<br>Fix integration bugs | Support Jayesh with<br>data format issues<br>Help test full pipeline | Wire webhook to Gemini<br>Test reply parsing<br>Support Jayesh with<br>WhatsApp integration |
| 6 | Deploy to Railway<br>Set environment variables<br>Final testing on live server | Help test live deployment<br>Verify API data is correct<br>on Railway | Prepare demo script<br>Test live demo 3 times<br>Practice viva answers |

## Key Milestones:

- **End of Week 2:** Rajveer shares sample data output with Harshita — she can now build real prompts
- **End of Week 4:** All individual modules are done and tested independently
- **Mid Week 5:** Jayesh connects everything — first full pipeline test
- **End of Week 5:** Full system works locally — all bugs fixed
- **End of Week 6:** Deployed on Railway — live demo ready

# 09 — Integration Checklist & Data Format

This is the single most important page for avoiding integration problems. ALL team members must agree on this data format by the end of Week 2.

**STANDARD DATA FORMAT (all modules must follow this)**

```
{
  "outage": [
    {"start": "10:00", "end": "12:00", "stage": 3},
    {"start": "18:00", "end": "20:00", "stage": 3}
  ],
  "solar": [
    {"hour": "06:00", "kwh": 0.0},
    {"hour": "12:00", "kwh": 1.8}
  ],
  "inverter": {
    "battery_pct": 78,
    "solar_watts": 0,
    "grid_on": true
  },
  "household": [
    {"name": "Geyser", "watts": 3000, "priority": 1,
     "deadline": "08:00", "flexible": false}
  ]
}
```

## Integration Checklist:

- Rajveer's get_schedule() returns a list of dicts with 'start', 'end', 'stage' keys
- Rajveer's get_solar_forecast() returns a list of dicts with 'hour', 'kwh' keys
- Rajveer's SimulatedInverter().get_snapshot() returns a dict with 'battery_pct', 'solar_watts', 'grid_on'
- Rajveer's SimulatedHousehold().get_appliances() returns a list of dicts with 'name', 'watts', 'priority', 'deadline', 'flexible'
- Harshita's generate_plan() accepts exactly 4 arguments: outage, solar, inverter, household
- Harshita's generate_plan() returns a plain string (the WhatsApp message text)
- Harshita's send_message(to, body) accepts a phone number and message text
- Harshita's handle_reply(message, plan) returns a response string
- Jayesh's /run-now endpoint triggers the full pipeline and returns a success/fail JSON
- Jayesh's /webhook endpoint accepts POST requests from Twilio

# 10 — GitHub Workflow & Team Rules

## Repository Setup:

- Jayesh creates one repo: `smart-scheduler`
- Add Rajveer and Harshita as collaborators (Settings > Collaborators)
- Create 3 branches: `backend` (Jayesh), `data` (Rajveer), `ai` (Harshita)
- Each person works ONLY on their branch
- Jayesh merges all branches into `main` during Week 5 integration

## .gitignore (create this file in root):

```
.env venv/ __pycache__/ *.pyc *.db .DS_Store
```

### TEAM RULES (follow these strictly!)

**Rule 1: FIX DATA FORMAT EARLY**
All outputs must follow the SAME format from Section 09. Agree on this by end of Week 2.

**Rule 2: USE GITHUB PROPERLY**
Commit daily. Write clear commit messages. Never push to main directly.

**Rule 3: DAILY 10-MINUTE SYNC**
Quick WhatsApp call or message every evening. Share progress, blockers, and next steps. This prevents "integration hell" in Week 5.

**Rule 4: TEST BEFORE YOU PUSH**
Every module should have a simple test script. Run it before pushing your code.

# 11 — Deployment on Railway

Jayesh handles deployment in Week 6. Here is the step-by-step process:

**Step 1**

Push your code to GitHub (create a free account if you don't have one)

**Step 2**

Go to railway.app and sign up with your GitHub account

**Step 3**

Click 'New Project' then 'Deploy from GitHub repo' then select your repo

**Step 4**

Railway auto-detects Python and installs requirements.txt

**Step 5**

Go to your project's Variables tab and add ALL your .env keys here

**Step 6**

Go to Settings and add Start Command: uvicorn api.main:app --host 0.0.0.0 --port $PORT

**Step 7**

Railway gives you a public URL like: https://smart-scheduler.up.railway.app

**Step 8**

Copy this URL, go to Twilio Sandbox settings, paste URL + /webhook as incoming message URL

**Step 9**

Your app is now live and will run the 6 AM job every day automatically!

## Cost on Railway

Railway gives $5 free credits per month. A small FastAPI app with SQLite uses roughly $0.50-1.00/month of compute. You will not exceed the free tier for a college project with 1-2 test households.

# 12 — Demo Script for Viva / Presentation

When you present to your professor or judges, follow this exact script. Practice it 3 times before the day.

## DEMO 1

**Open WhatsApp**

Show the judge a real phone with WhatsApp open on the test number. This immediately shows your project is real and working.

## DEMO 2

**Trigger the Plan Manually**

Hit the /run-now endpoint in your browser. The WhatsApp message arrives on the phone within seconds. This is the 'wow moment'.

## DEMO 3

**Read the Plan Aloud**

Let the judge read the plan. Point out: 'This used real EskomSePush data for today's schedule and real Open-Meteo solar data.'

## DEMO 4

**Reply to WhatsApp**

Type a reply: 'Change washing machine to 4 PM'. Show Gemini parsing it and sending a confirmation back within a few seconds.

## DEMO 5

**Show the Code Briefly**

Show simulators/inverter.py and agent/scheduler.py. Explain: 'Hardware is simulated — this class would be replaced by a real inverter API.'

## DEMO 6

**Show Railway Dashboard**

Show the Railway dashboard with your live deployment and the scheduled 6 AM job. This proves it runs automatically.

## VIVA STATEMENTS

### One-line summary for your showcase:

*"We built an AI energy advisor that pulls real South African load shedding schedules, reasons over them with Gemini, and sends households an optimised daily appliance plan over WhatsApp — completely free to run, no hardware required."*

### For the viva, you can say:

*"We divided the system into three layers — data acquisition, AI reasoning, and backend orchestration. Rajveer handled real and simulated data inputs, Harshita built the AI planning and WhatsApp interaction, and Jayesh integrated everything into a FastAPI backend with scheduling and deployment."*